



VIT[®]

BHOPAL

PROJECT REPORT

AI Sentiment Analyser

NAME: YAANA RAJPUT

REG NO. 25BCE10680

COURSE: FUNDAMENTALS OF AI AND ML
(CSA2001)

FACULTY : DR. SATYENDRA SINGH

SLOT : B21+E14+E22

DATE: 31/03/2026

INTRODUCTION

I decided to build an AI Sentiment Analyser. Nowadays, people post so much on social media and it is hard to keep track of whether the comments are good or bad. I wanted to create a simple app where you can type in any sentence and the computer tells you if it sounds "Positive" or "Negative."

To make this work, I used Python. Which is one of the most popular programming languages. I also used a tool called Streamlit to turn my code into a real website that anyone can use in a browser. The "brain" of my project is a library called NLTK, which helps the computer understand human emotions in text.

PROBLEM STATEMENT

Nowadays, businesses and apps get thousands of reviews and comments every minute. It is impossible for a human to read every single one to see if people are happy or angry. We need a way for a computer to "read" these messages and automatically sort them.

The problem is that human language is tricky—we use slang, emojis, and capital letters to show feeling. This project solves that by building a tool that can instantly read a sentence and give it a "Sentiment Score" to show if it's Positive or Negative.

OBJECTIVE

- Create a Web Interface: Use Streamlit so users can type into a clean website instead of a boring black coding screen.
- Use AI Logic: Use a library called NLTK (VADER) which is pre-trained to understand the "weight" of words (for example, it knows "excellent" is more positive than "good").
- Real-Time Results: Make sure the AI gives an answer the second the user clicks the "Analyse" button.
- Learn the Workflow: Learn how to manage a Python project, handle errors in the terminal, and save code to GitHub.

Methodology and Approach

To develop the AI Sentiment Analyser, I followed a structured software development process. The approach is divided into four main phases:

A. Environment Setup

- 1) Editor: I used Visual Studio Code (VS Code) as the primary Integrated Development Environment (IDE).
- 2) Language: The project was developed using Python 3.x.
- 3) Libraries: I installed the necessary dependencies using the Python Package Manager (pip):-

streamlit: For creating the web interface.

nltk: To access the Natural Language Toolkit and the VADER sentiment model.

B. Core Logic Development

1) Data Import: I imported the SentimentIntensityAnalyzer from the NLTK library.

2) Model Initialization: I used the `@st.cache_resource` function to load the AI model into the computer's memory so it runs faster.

3) Algorithm: The system uses the VADER lexicon, which assigns a numerical "valence" score to words (e.g., "happy" = 2.7, "sad" = -2.1).

C. User Interface (UI) Design

1) Input Handling: I created a `st.text_area` where users can type their messages.

2) Trigger Mechanism: I added a "Analyze" button using `st.button`.

3) Output Display: I wrote logic using if-else statements to show a Green success box if the score is positive and a Red error box if the score is negative.

D. Execution and Deployment

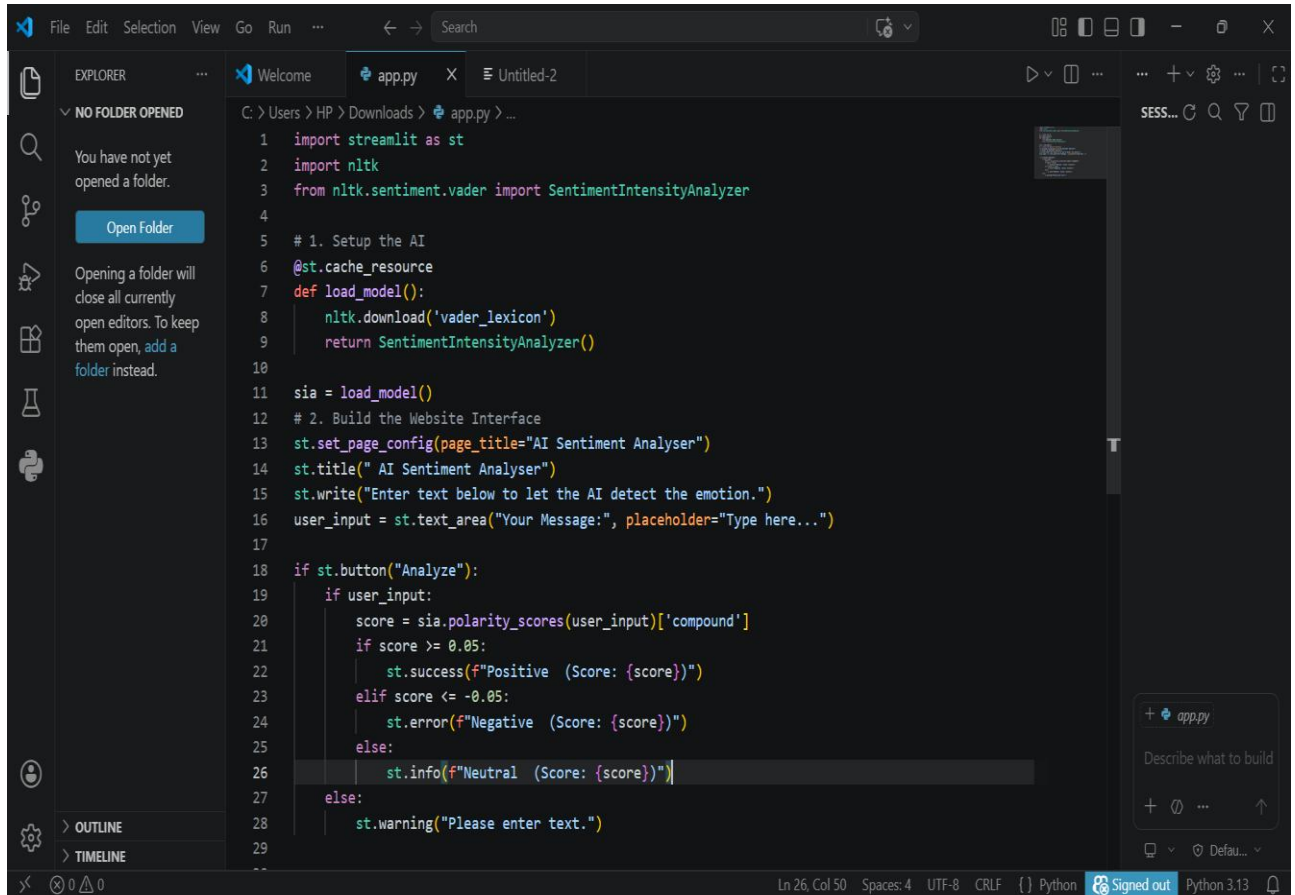
- 1) Directory Navigation: I used the terminal command `cd Downloads` to ensure the system was looking in the correct folder for the `app.py` file.
- 2) Local Hosting: I launched a local web server using the command:
`python -m streamlit run app.py`
- 3) Testing: I performed several tests with different sentences to verify that the AI was correctly identifying emotions.

Key Decisions & Design Choices

- 1) Logic (VADER): Chose a rule-based AI engine because it is fast, lightweight, and specifically understands social media language (slang, emojis, and punctuation).
- 2) Interface (Streamlit): Selected for its ability to build web apps using only Python, which kept the development process simple and efficient.
- 3) Metric (Compound Score): Used a single score (from -1 to 1) to make the final results easy for the user to understand.
- 4) Execution: Opted for a local server (localhost) to master directory navigation and terminal-based deployment.

Screenshots and Result

INPUT:

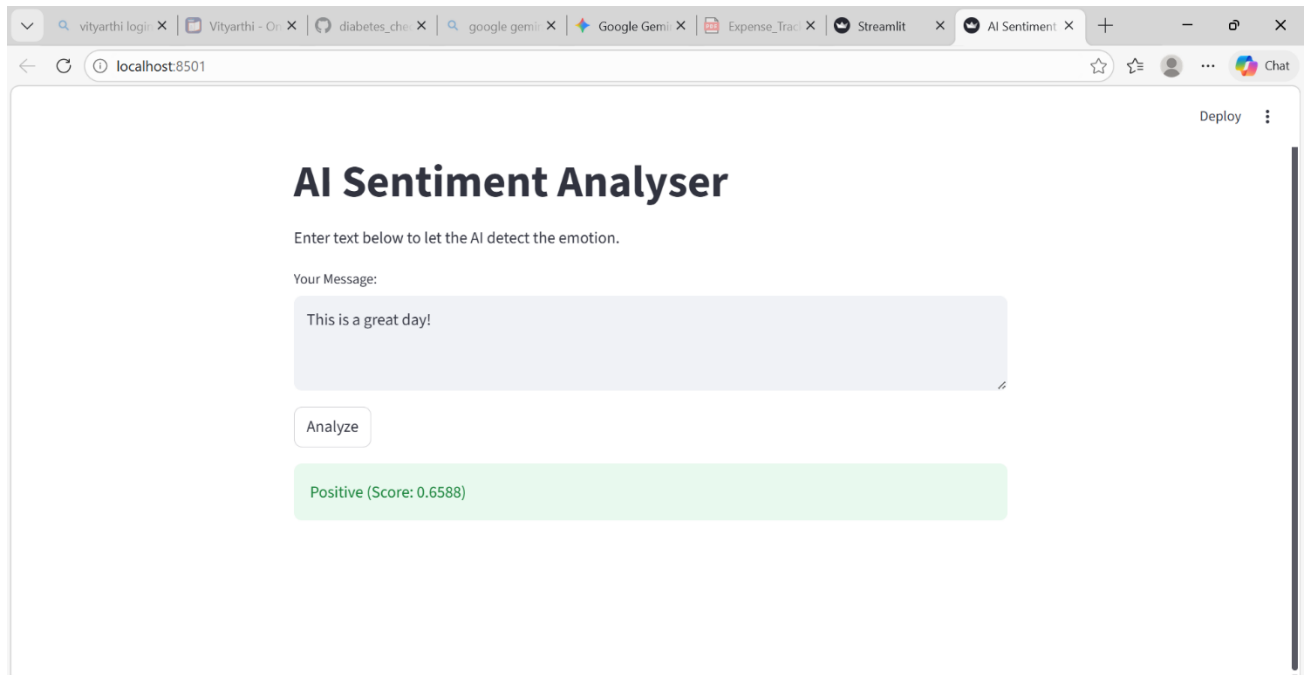


The screenshot displays a Visual Studio Code editor window with a Python file named `app.py` open. The file path is `C:\Users\HP\Downloads\app.py`. The code implements a sentiment analysis application using Streamlit and NLTK's VaderSentiment.

```
1 import streamlit as st
2 import nltk
3 from nltk.sentiment.vader import SentimentIntensityAnalyzer
4
5 # 1. Setup the AI
6 @st.cache_resource
7 def load_model():
8     nltk.download('vader_lexicon')
9     return SentimentIntensityAnalyzer()
10
11 sia = load_model()
12 # 2. Build the Website Interface
13 st.set_page_config(page_title="AI Sentiment Analyser")
14 st.title(" AI Sentiment Analyser")
15 st.write("Enter text below to let the AI detect the emotion.")
16 user_input = st.text_area("Your Message:", placeholder="Type here...")
17
18 if st.button("Analyze"):
19     if user_input:
20         score = sia.polarity_scores(user_input)['compound']
21         if score >= 0.05:
22             st.success(f"Positive (Score: {score})")
23         elif score <= -0.05:
24             st.error(f"Negative (Score: {score})")
25         else:
26             st.info(f"Neutral (Score: {score})")
27     else:
28         st.warning("Please enter text.")
```

The interface on the right side of the editor shows a Streamlit session state with a text input field and a button labeled "Analyze". The status bar at the bottom indicates the file is using Python 3.13.

OUTPUT:



CONCLUSION

To conclude, this project successfully shows how to build a working AI Sentiment Analyser using Python. By connecting the VADER AI model with the Streamlit web tool, I created an app that can instantly understand if a sentence is positive or negative.

The main lessons I learned from this project are:

- Connecting AI to a Website: I learned how to take a "thinking" model (NLTK) and give it a real interface that users can interact with.

- Solving Technical Problems: I improved my skills in using the terminal and fixing errors related to file paths and folders.
- Speed of Automation: I saw how a simple computer program can analyze text much faster and more accurately than a human doing it manually.

Overall, this project gave me a great start in Natural Language Processing (NLP). It proves that with the right Python libraries, we can build smart tools that help us understand human emotions in digital data.

References:

1) Python Language

- Source: Python Software Foundation
- Link: <https://www.python.org/>

2) Streamlit Framework

- Source: Streamlit Documentation
- Link: <https://docs.streamlit.io/>

3) NLTK Library

- Source: Natural Language Toolkit
- Link: <https://www.nltk.org/>

4) VADER Sentiment Model

- Source: C.J. Hutto (GitHub Research)
- Link: <https://github.com/cjhutto/vaderSentiment>

5) Visual Studio Code

- Source: Microsoft Code Documentation
- Link: <https://code.visualstudio.com/>

